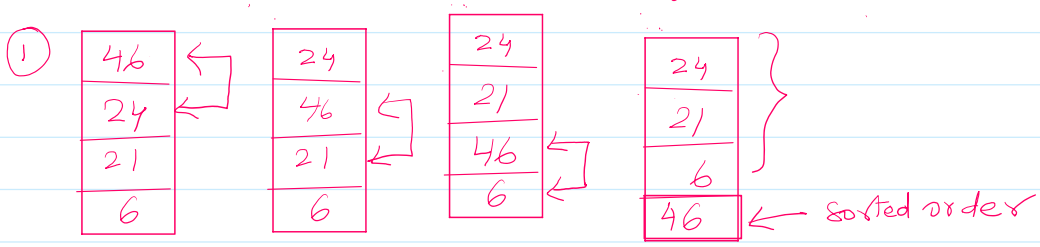
Insertion Sort

13 October 2025 20:23

In bubble sort we compare adjacent elements and swap elements as per given condition. After comparing all the elements, the last element is in sorted order. In every pass 1-element is in order.



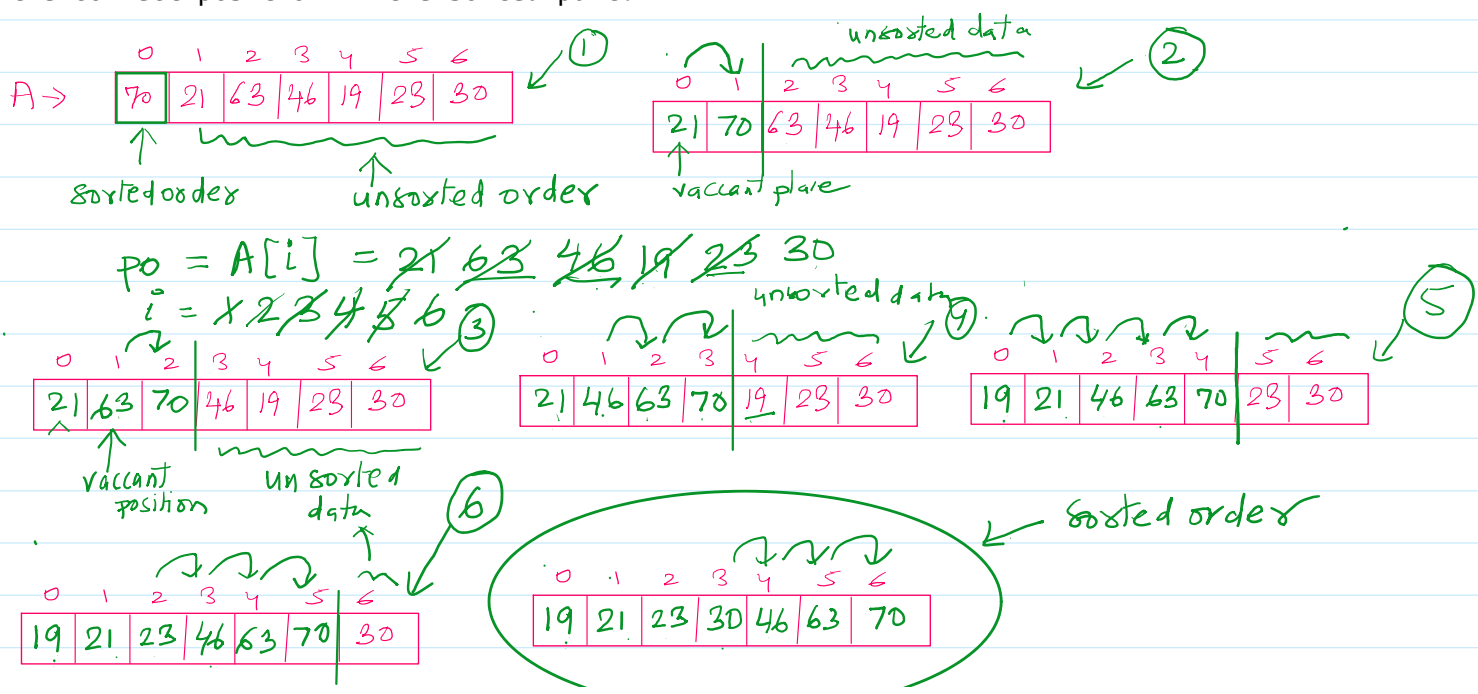
Performance of bubble sort is $O(n^2)$ where 'n' is number of elements.

Introduction to Insertion Sort (Natural Sort)
Insertion sort is a comparison based sorting algorithm that works the same way you would sort playing cards in your hand.\

Real-life Analogy

Imagine you are holding playing cards and insulting each new card into its correct position among the already sorted ones in your hand.
That's exactly what insertion sort does with numbers. (or array elements).

Working Principle
Insertion sort divides the array into two parts.
1. **Sorted part**(Initially contains the first element)
2. **Unsorted part** (The rest of the elements)
Then it repeatedly picked the first element from the unsorted part and insert it into the correct position in the sorted part.



0	1	2	3	4	5	6
19	21	23	46	63	70	30

19	21	23	30	46	63	70

The internal steps or inner loop also executes for 'n' number of times. And, the outer loop also executes for 'n' number of times. Therefore the performance of insertion sort is $O(n^2)$, which is similar to bubble sort and selection sort.

Algorithm Steps.

1. Start from second element (index 1), because the 1st element is already sorted.
2. Store the current element in a variable (say key).
3. Compare key with elements on the left of it (sorted portion)
4. Shift all the elements greater than 'key' one position to the right.
5. Insert the 'key' at its correct position.
6. Repeat until the array is fully sorted.

```
public class Inserting_Sort
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter The Number of Elements : ");
        int n = sc.nextInt();
        int A[] = new int[n];
        int i =1, j;
        for(int j =0; j<n; j++){
            System.out.print("\nEnter A Number : ");
            A[j] = sc.nextInt();
        }
        for(;i<n;i++){
            for(key = A[i], j=i-1; j>=0 && A[j]>key; j--){
            A[j+1] = A[j];
            }
            A[j+1] = key;
        }
        for(i=0;i<n;i++){
            System.out.print(A[i] + " ");
        }
    }
}
```

n = 7

21 63

F

i = ~~1~~ 2

key = ~~21~~ 63

j = ~~0~~ ~~-1~~ ~~0~~

maze

0	1	2	3	4	5	6
21	63	70	46	19	23	30

L

Introduction to backtracking Algorithm

Introduction to backtracking Algorithm.